

UNIT 3: Networking Basics

1. Protocols: The Foundation of Network Communication

Definition

A **protocol** is a meticulously defined, formal set of rules, conventions, and procedures that govern how data is formatted, transmitted, and received across a network. It's the essential "common language" that enables diverse devices to communicate and interact seamlessly. Without standardized protocols, global networking as we know it would be impossible.

These rules dictate:

- **Data Formatting and Structure:** Protocols define the syntax of data, including the precise arrangement of bits and bytes, the fields within headers (like source/destination addresses), and the actual data payload. This structured approach ensures that a receiving device can correctly parse and interpret the incoming stream of information.
- **Transmission Mechanisms:** They specify the timing, synchronization, and ordering of data units (e.g., bits, frames, packets). This includes aspects like bit rates, modulation techniques (at the physical layer), and mechanisms for flow control to prevent a fast sender from overwhelming a slow receiver.
- **Reception and Processing Procedures:** Protocols detail how the receiving device acknowledges successful receipt of data, detects and handles errors (e.g., corruption, loss), and reassembles fragmented data back into its original form.

Why Protocols are Indispensable

Protocols are absolutely crucial for network functionality and scalability. They provide:

- **Interoperability:** They enable hardware and software from different vendors to communicate. Imagine if every phone brand used a different charging port – protocols solve this by standardizing communication interfaces and rules.
- **Reliability:** Many protocols include mechanisms for error detection, correction, and retransmission, ensuring that data arrives intact and in the correct order.
- **Efficiency:** By defining clear procedures, protocols optimize the use of network resources, preventing chaos and collisions.
- **Scalability:** The layered nature of protocols (discussed below) allows for the independent development and evolution of different network components, supporting massive network growth.
- **Troubleshooting:** The modularity provided by protocols simplifies the process of identifying and isolating network issues to a specific layer or component.

Common Networking Protocols: A Deeper Dive

Here's an expanded look at some of the most frequently encountered networking protocols:

PROTOCOL	FUNCTION
----------	----------

DNS	The Domain Name System translates human-friendly domain names (e.g., www.example.com) into numerical IP addresses (e.g., 192.0.2.1). This is crucial for locating services on the internet, as devices communicate using IP addresses. It operates primarily over UDP for speed. How it works: When you type a URL, your computer sends a DNS query to a DNS server. If the server knows the IP, it replies; otherwise, it queries other DNS servers (a hierarchical process) until the IP is found. The result is then cached for faster future lookups.
DHCP	The Dynamic Host Configuration Protocol automatically assigns IP addresses and other crucial network configuration details (like subnet mask and DNS servers) to devices on a network. This significantly reduces manual setup burden and prevents IP address conflicts. How it works (DORA process): Discover: A client broadcasts a DHCP Discover message. Offer: DHCP servers respond with an offer. Request: The client requests an IP. Acknowledge: The server confirms the lease.
ICMP	The Internet Control Message Protocol is primarily used for network diagnostics and error reporting. Devices like routers use it to send essential messages about issues (e.g., unreachable hosts, too much traffic). It underpins critical tools like ping and traceroute. Key messages: Echo Request/Reply (Ping): Tests reachability. Destination Unreachable: Router cannot deliver a packet. Time Exceeded: Packet's TTL reached zero.
SSH	Secure Shell is a cryptographic network protocol for secure remote login and command-line execution over an unsecured network. It uses strong encryption to create a secure channel and is also widely used for secure file transfers (via SFTP - SSH File Transfer Protocol and SCP - Secure Copy Protocol). It provides mutual authentication (both client and server verify each other) and encrypts all communications, making it ideal for remote administration and management of servers and network devices.

- **DNS (Domain Name System)**

- **Function:** A fundamental, global decentralized system used to translate human-readable hostnames (like www.example.com) into numerical IP addresses (e.g., 192.0.2.1). This is essential because computers communicate using IP addresses, while humans prefer names. DNS is often called the "phonebook of the internet."
- **How it works:** When you type a URL, your computer sends a DNS query to a DNS server. If the server knows the IP, it replies; otherwise, it queries other DNS servers (a hierarchical process) until the IP is found. The result is then cached for faster future lookups.
- **Key aspects:**

- **Hierarchical structure:** .com, .org, .in are Top-Level Domains (TLDs). Then come second-level domains (e.g., google.com).
- **Distributed database:** No single server holds all DNS information; it's spread across millions of DNS servers worldwide.
- **Caching:** To improve performance, DNS resolvers store resolved IP addresses for a period, reducing the need for repeated queries.
- **Example Use Cases:**
 - Typing any website URL into a browser.
 - Email servers finding the correct recipient domain's mail server.
 - Any network service that references domain names (e.g., connecting to a cloud service by name).
- **DHCP (Dynamic Host Configuration Protocol)**
 - **Function:** An application layer protocol that automates the process of assigning IP addresses and other crucial network configuration parameters (like subnet mask, default gateway, DNS server addresses) to devices on a network. This eliminates the need for manual configuration and drastically reduces potential IP address conflicts.
 - **How it works (DORA process):**
 - **Discover:** A client broadcasts a DHCP Discover message to find a DHCP server.
 - **Offer:** DHCP servers on the network respond with a DHCP Offer, proposing an IP address and configuration.
 - **Request:** The client selects an offer and sends a DHCP Request, formally requesting the IP address.
 - **Acknowledge:** The chosen DHCP server sends a DHCP ACK, confirming the IP address lease and providing the full configuration.
 - **Benefits:** Simplifies network administration, reduces errors, supports mobile users, and allows efficient IP address reuse.
 - **Example Use Cases:**
 - A laptop connecting to Wi-Fi at home, in a coffee shop, or at work.
 - Smartphones, tablets, and IoT devices automatically getting network access.
 - Automatic assignment of IPs in large corporate networks and data centers.
 - Home routers providing IP addresses to all connected devices.
- **ICMP (Internet Control Message Protocol)**

- **Function:** A network layer (Layer 3) protocol used primarily for network diagnostics, error reporting, and operational messages. It's not designed to carry user data, but rather to communicate information about network problems back to the source.
- **Key messages:**
 - **Echo Request/Reply (Ping):** Used to test reachability and round-trip time to a host.
 - **Destination Unreachable:** Sent when a router cannot deliver a packet to its destination (e.g., network unreachable, host unreachable, port unreachable).
 - **Time Exceeded:** Sent when a packet's TTL reaches zero (used by traceroute/tracert to map network paths).
 - **Redirect:** Informs a host about a better route to a particular destination.
- **Importance:** Critical for network administrators to troubleshoot connectivity issues, identify bottlenecks, and diagnose routing problems.
- **Example Use Cases:**
 - Using the ping command to check if a server is online and responsive.
 - Running traceroute or tracert to map the path packets take across the internet and identify where delays or failures occur.
 - Network monitoring systems using ICMP to detect downed hosts or network segments.
- **SSH (Secure Shell)**
 - **Function:** A cryptographic network protocol for secure remote login and command-line execution over an unsecured network. It provides a secure channel over an unsecured network by using strong encryption.
 - **Key features:**
 - **Secure remote access:** Replaces insecure protocols like Telnet.
 - **Secure file transfer:** Supports SFTP (SSH File Transfer Protocol) and SCP (Secure Copy Protocol).
 - **Port forwarding/Tunneling:** Creates encrypted tunnels for other services, enhancing security for otherwise unencrypted traffic.
 - **Authentication:** Supports password, public key, and host-based authentication.
 - **Architecture:** Uses a client-server model. The SSH client connects to the SSH server on the remote machine.
 - **Example Use Cases:**

- Remotely managing servers via command line from anywhere in the world.
- Securely uploading or downloading files to/from web servers or cloud instances.
- Configuring network devices (routers, switches) securely.
- Creating an encrypted tunnel to securely browse the internet from a public Wi-Fi network.

2. Packets: The IP Header – The Routing Blueprint

Packet Definition

A **packet** is the fundamental, self-contained unit of data transmitted over an IP network. When any data (an email, a streaming video segment, a web page component) is sent across a network, it's broken down into these small, manageable chunks. Each packet contains a portion of the original data (the payload) along with crucial control information (headers) that allows it to travel independently across various network devices and be correctly reassembled at its final destination.

IP Header Fields: An Exhaustive Breakdown

The **IP (Internet Protocol) header** is the essential control section of an IP packet, preceding the actual data. Its primary purpose is to enable routing packets across the Internet.

- **Version (4 bits):**
 - **Purpose:** Identifies the version of the Internet Protocol being used.
 - **Values:** 0100 for IPv4, 0110 for IPv6.
 - **Impact:** Crucial for routers and hosts to correctly interpret the structure and content of the rest of the IP header. If a device expects IPv4 but receives an IPv6 packet (or vice versa), it won't be able to process it correctly.
- **Internet Header Length (IHL) (4 bits):**
 - **Purpose:** Specifies the length of the IP header itself.
 - **Units:** Measured in 32-bit words (4-byte chunks).
 - **Details:** The minimum IHL is 5 ($5 * 4 \text{ bytes} = 20 \text{ bytes}$), which is the length of a standard IPv4 header with no options. If IP options are present, the IHL will be greater than 5. A maximum value of 15 ($15 * 4 \text{ bytes} = 60 \text{ bytes}$) means the IP header can be up to 60 bytes long.
- **Type of Service (ToS) / Differentiated Services Code Point (DSCP) (8 bits):**
 - **Purpose:** Originally named ToS, it's now predominantly used for **Differentiated Services (DiffServ)** through the DSCP field. This field allows network devices (e.g., routers) to classify and potentially prioritize traffic based on its type.

- **Functionality:**
 - **Quality of Service (QoS):** Critical for applications requiring specific performance guarantees, such as real-time voice (VoIP) or video conferencing, which might receive higher priority than bulk data transfers (like file downloads).
 - **Congestion Management:** Routers can use DSCP values to manage congestion by, for example, dropping lower-priority packets first.
- **Total Length (16 bits):**
 - **Purpose:** Defines the total length of the entire IP packet, encompassing both the IP header and the data payload.
 - **Units:** Measured in bytes.
 - **Details:** Since it's a 16-bit field, the maximum possible IP packet size is $2^{16}-1=65,535$ bytes. However, practical limits are often much smaller due to the **Maximum Transmission Unit (MTU)** of underlying network technologies (e.g., Ethernet's typical MTU is 1500 bytes), leading to fragmentation if packets exceed this size.
- **Identification (16 bits):**
 - **Purpose:** A unique value assigned to each IP packet by the sender.
 - **Functionality:** Essential for **fragmentation and reassembly**. If a packet needs to be broken into smaller pieces (fragments) to traverse a network link with a smaller MTU, all fragments of the *original* packet will share the same Identification number. This allows the receiving host to correctly group and reassemble them.
- **Flags (3 bits):**
 - **Purpose:** Control bits primarily related to IP fragmentation.
 - **Breakdown:**
 - **Bit 0 (Reserved):** Always set to 0.
 - **Bit 1 (DF - Don't Fragment):**
 - If 1: The packet should *not* be fragmented. If a router encounters a link with an MTU smaller than the packet size, and this flag is set, the packet will be dropped. An ICMP "Fragmentation Needed" (Type 3, Code 4) error message is then sent back to the source.
 - If 0: Fragmentation is allowed.
 - **Bit 2 (MF - More Fragments):**
 - If 1: Indicates that this is *not* the last fragment of a fragmented packet; more fragments are expected to follow.

- If 0: Indicates that this is either the last fragment of a fragmented packet or the packet was not fragmented at all (it's the only fragment).
- **Fragment Offset (13 bits):**
 - **Purpose:** Specifies the offset of the current fragment's data relative to the beginning of the original, unfragmented data payload.
 - **Units:** Measured in units of 8 bytes (64 bits). This means the actual offset is Fragment Offset * 8.
 - **Functionality:** Used by the receiving host, in conjunction with the Identification field and Flags, to correctly reassemble fragmented packets in their original order.
- **Time to Live (TTL) (8 bits):**
 - **Purpose:** A counter that prevents IP packets from looping indefinitely on the network, consuming resources and never reaching their destination.
 - **Mechanism:** Each time a packet passes through a router (a "hop"), the router decrements the TTL value by 1.
 - **Action at Zero:** When the TTL value reaches 0, the packet is discarded by the router, and an ICMP "Time Exceeded" (Type 11, Code 0) message is typically sent back to the source IP address.
 - **Common Initial Values:** Operating systems set different default TTL values (e.g., 64 for Linux, 128 for Windows, 255 for Cisco routers). This can sometimes reveal the OS of the source device.
- **Protocol (8 bits):**
 - **Purpose:** Identifies the next-level protocol encapsulated within the IP packet's payload. This tells the receiving host which higher-layer protocol handler should receive and process the data after the IP header has been removed.
 - **Common Values (IANA Assigned Protocol Numbers):**
 - 6: **TCP** (Transmission Control Protocol)
 - 17: **UDP** (User Datagram Protocol)
 - 1: **ICMP** (Internet Control Message Protocol)
 - 89: **OSPF** (Open Shortest Path First)
 - 47: **GRE** (Generic Routing Encapsulation)
 - **Importance:** Essential for multiplexing and demultiplexing data to the correct transport layer protocol.
- **Header Checksum (16 bits):**
 - **Purpose:** Used solely for **error detection** within the IP header itself. It helps ensure the integrity of the header information.

- **Mechanism:** Calculated by the sender. Critically, it is *recalculated* by *every router* that modifies the IP header (e.g., when decrementing the TTL).
- **Action on Mismatch:** If the checksum doesn't match upon receipt (indicating corruption of the header), the packet is typically discarded without processing.
- **Note:** This checksum *does not* cover the data payload; integrity of the payload is usually handled by higher-layer protocols (e.g., TCP checksum).
- **Source IP Address (32 bits):**
 - **Purpose:** The 32-bit (4-octet) IP address of the originating sender of the packet. This is the address of the device that initially created and sent the packet.
- **Destination IP Address (32 bits):**
 - **Purpose:** The 32-bit (4-octet) IP address of the intended final receiver of the packet. This is the address that routers use to determine the next hop for forwarding the packet.
- **Options (Variable, Optional):**
 - **Purpose:** An optional field used for various functions that are not part of the standard IP header.
 - **Usage:** Not commonly seen in typical internet traffic due to performance overhead and security implications. Primarily used for network testing, debugging, or specific security features.
 - **Examples:**
 - **Record Route:** Requests that each router along the path records its IP address in the header.
 - **Timestamp:** Requests that each router record the time it processed the packet.
 - **Source Routing:** Allows the sender to specify a strict or loose path the packet must take.
 - **Length:** The length of the options field is indicated by the IHL field.
- **Padding (Variable, Optional):**
 - **Purpose:** Added to ensure that the IP header always ends on a 32-bit (4-byte) boundary. This is necessary because some network hardware and software implementations are optimized for processing data in 32-bit chunks.
 - **Mechanism:** If the total length of the IHL and Options field does not perfectly align to a multiple of 4 bytes, null (zero) bits are added as padding until it does.

3. Protocol Layers: The TCP/IP Model (A Practical Perspective)

Context: The Need for Layering

Networking is inherently complex. To manage this complexity, network communication is divided into logical, modular layers. Each layer is responsible for a specific set of functions and interacts only with the layers immediately above and below it. This **layering** provides several benefits:

- **Modularity:** Changes to one layer don't necessarily affect others.
- **Simplicity:** Each layer handles a distinct set of tasks, making design and implementation easier.
- **Interoperability:** Different vendors can develop components for different layers, as long as they adhere to the interface definitions.
- **Troubleshooting:** Problems can be isolated to a specific layer, simplifying diagnosis.

The **TCP/IP model** is a pragmatic, functional model that largely underpins the modern Internet. While the OSI model is a theoretical framework, TCP/IP focuses on the protocols actually used. It's often depicted with 4 or 5 layers, combining some OSI layers for practical implementation.

4. ISO/OSI Model (7 Layers): The Theoretical¹ Framework

Context

While the simplified TCP/IP model describes the Internet's practical operation, the **OSI (Open Systems Interconnection) model** is a universal, conceptual framework developed by the International Organization for Standardization (ISO). It divides network communication into seven distinct layers, providing a standardized way to understand and discuss network functions, independent of specific technologies. It helps to understand *what* each part of the network is supposed to do, even if TCP/IP often combines these functions into fewer layers.

Benefits

The strict layering of the OSI model offers significant advantages:

- **Standardization:** Promotes open systems where products from different vendors can interoperate.
- **Modularity:** Allows developers to specialize in specific layers without needing to understand the entire network stack.
- **Easier Troubleshooting:** By isolating functions, problems can be more easily localized to a specific layer.
- **Flexibility:** New protocols and technologies can be developed for individual layers without disrupting the entire model.
- **Learning Tool:** Provides a clear and structured way to learn and teach networking concepts.

20. TCP and UDP Headers: The Transport Layer's Blueprint

Context

TCP (Transmission Control Protocol) and **UDP (User Datagram Protocol)** are the two foundational protocols of the **Transport Layer (Layer 4)**. They sit directly above the IP (Internet) layer and provide different types of services to applications, dictating how data is transported between end-user applications on different hosts. Understanding their headers reveals their fundamental differences in functionality and overhead.

TCP (Transmission Control Protocol)

TCP is a **connection-oriented** and **reliable** transport protocol. It's designed for applications where data integrity, order, and full delivery are paramount.

- **Key Features:**
 - **Connection-oriented:** Before any data is exchanged, TCP establishes a logical, full-duplex connection between the sender and receiver. This involves a **three-way handshake**:
 1. **SYN (Synchronize):** Sender sends a SYN packet to initiate the connection.
 2. **SYN-ACK (Synchronize-Acknowledgment):** Receiver responds with a SYN-ACK, acknowledging the SYN and synchronizing its own sequence number.
 3. **ACK (Acknowledgment):** Sender sends an ACK, acknowledging the SYN-ACK, completing the handshake. This ensures both ends are ready to communicate and establishes initial sequence numbers.
 - **Reliability:** TCP guarantees that data will arrive at the destination:
 - **In order:** Segments are reassembled in their original sequence using **Sequence Numbers**.
 - **Without errors:** Through checksums and retransmissions.
 - **Without loss:** Missing segments are detected via gaps in sequence numbers or unacknowledged data, triggering **Retransmission** by the sender.
 - **Flow Control:** Prevents a fast sender from overwhelming a slower receiver. The receiver advertises a **window size** (the amount of data it's currently willing to accept) in its acknowledgments. The sender must pause transmission if the window closes.
 - **Congestion Control:** Adapts the transmission rate dynamically to perceived network congestion (e.g., detected by lost acknowledgments or increased RTT), preventing TCP from exacerbating network overload.
 - **Full-duplex communication:** Data can be sent and received simultaneously over the same connection.
- **Performance:** Due to the significant overhead of connection setup, acknowledgments, retransmissions, and complex flow/congestion control algorithms, TCP is generally **slower** and more resource-intensive than UDP.

- **Typical Use Cases:** Any application where data integrity and guaranteed delivery are critical.
 - **HTTP/HTTPS (Web Browse):** You want web pages to load completely and correctly.
 - **FTP (File Transfer Protocol):** Files must be transferred without corruption.
 - **SMTP (Simple Mail Transfer Protocol):** Emails must arrive intact.
 - **SSH (Secure Shell):** Secure remote terminal access requires reliable command execution.
 - **Database Connections:** Ensuring data consistency.

UDP (User Datagram Protocol)

UDP is a **connectionless** and **unreliable** (best-effort) transport protocol. It's a minimalistic protocol, providing speed and efficiency at the cost of reliability.

- **Key Features:**
 - **Connectionless:** UDP does not establish a connection before sending data. It simply sends **datagrams** (UDP's unit of data) without any prior coordination. It's often described as a "fire and forget" protocol.
 - **Unreliability (Best-Effort Delivery):** UDP offers no guarantees regarding:
 - **Delivery:** Datagrams may be lost.
 - **Order:** Datagrams may arrive out of sequence.
 - **Error Checking:** Beyond a basic checksum for the UDP header and payload, UDP itself does not provide robust error recovery; lost or corrupted datagrams are *not* retransmitted by UDP.
 - **Minimal Overhead:** With no connection setup, no acknowledgments, no retransmissions, and no flow/congestion control, UDP is much **faster** and more efficient in terms of network overhead and processing power.
 - **No Flow/Congestion Control:** This can be both a strength (for real-time apps) and a weakness (can flood a congested network).
- **Performance:** UDP is preferred when speed and low latency are more important than guaranteed delivery, or when reliability is handled by the application layer itself.
- **Typical Use Cases:** Applications tolerant of occasional data loss or those that implement their own reliability mechanisms.
 - **DNS (Domain Name System):** Quick queries are essential. If a DNS query fails, the application simply retries.
 - **VoIP (Voice over IP):** A slight delay or lost packet is less impactful than retransmitting and delaying an entire conversation.

- **Streaming Video/Audio:** Losing a few frames is usually preferable to a complete freeze while lost data is retransmitted.
- **Online Gaming:** Latency is critical; retransmitting old game state data is usually undesirable and can cause lag.
- **SNMP (Simple Network Management Protocol):** Used for simple network device management where individual query/response failures are not critical.

TCP Header Fields (Detailed): (Minimum 20 bytes, up to 60 bytes with options)

The TCP header is significantly more complex than UDP's due to its robust features.

- **Source Port (16 bits):** Identifies the port number of the application process on the sending host. This allows the receiving host to know which application sent the data.
- **Destination Port (16 bits):** Identifies the port number of the application process on the receiving host. This allows the receiving host to deliver the data to the correct application.
- **Sequence Number (32 bits):** This is a crucial field for reliability and ordering. It represents the byte number of the first byte of data in the current segment relative to the beginning of the entire byte stream for that connection. Used to reassemble data in the correct order and detect missing segments.
- **Acknowledgment Number (32 bits):** Used by the receiver to inform the sender of the next sequence number it expects to receive. It acknowledges successful receipt of all bytes up to Acknowledgment Number - 1.
- **Data Offset (4 bits):** Also known as "Header Length." Specifies the length of the TCP header in 32-bit words (4-byte units). Similar to the IP Header Length, this field indicates where the data payload begins. Minimum value is 5 (20 bytes), maximum is 15 (60 bytes).
- **Reserved (6 bits):** Reserved for future use and typically set to zero.
- **Flags (Control Bits) (6 bits):** These are single-bit flags that control the connection state and specific actions:
 - **URG (Urgent Pointer Significant):** If set, the Urgent Pointer field is valid.
 - **ACK (Acknowledgment Significant):** If set, the Acknowledgment Number field is valid. This flag is almost always set after the initial SYN.
 - **PSH (Push Function):** Tells the receiving application to immediately "push" all buffered data up to the application layer without waiting for a full buffer.
 - **RST (Reset Connection):** Forces the termination of a connection, usually due to an error or an abnormal situation.
 - **SYN (Synchronize Sequence Numbers):** Used to initiate a connection (first step of the three-way handshake).
 - **FIN (Finish No More Data):** Used to gracefully terminate a connection, indicating that the sender has no more data to send.

- **Window Size (16 bits):** Used for **flow control**. The receiver advertises the amount of buffer space (in bytes) it has available to receive data. This tells the sender how much data it can send before it must wait for an acknowledgment. A zero window size means the sender must stop.
- **Checksum (16 bits):** A mandatory field for **error detection**. It's calculated over the TCP header, the TCP payload, and a pseudo-header derived from the IP header (source/destination IP, protocol, TCP length). If the checksum calculated by the receiver doesn't match, the segment is discarded.
- **Urgent Pointer (16 bits):** If the URG flag is set, this field indicates an offset from the Sequence Number to the last byte of urgent data. This allows urgent data to be processed out-of-band.
- **Options (Variable, Optional):** Provides extended functionality. Common options include:
 - **MSS (Maximum Segment Size):** Specifies the largest segment a host can receive.
 - **Window Scale:** Allows the Window Size field to be scaled to larger values for high-bandwidth networks.
 - **SACK (Selective Acknowledgments):** Allows the receiver to acknowledge specific blocks of data it received, even if there are gaps (missed segments) in between.
- **Padding (Variable, Optional):** Used to ensure the TCP header ends on a 32-bit boundary, similar to IP header padding.

UDP Header Fields (Detailed): (Fixed 8 bytes)

The UDP header is remarkably simple, reflecting its "fire and forget" nature.

- **Source Port (16 bits):** Identifies the port number of the application process on the sending host. (Optional; can be 0 if not used).
- **Destination Port (16 bits):** Identifies the port number of the application process on the receiving host. Essential for delivering the datagram to the correct application.
- **Length (16 bits):** The total length of the UDP header and the UDP data payload in bytes. The minimum length is 8 bytes (just the header). This field allows the receiver to know the exact size of the UDP datagram.
- **Checksum (16 bits):** Used for optional **error detection** across the UDP header and payload. In IPv4, this field can be zero if the checksum is not calculated. However, it is **mandatory in IPv6** (though the actual calculation might result in zero). If calculated, it helps detect corruption but does not trigger retransmission.